

EC-341 Digital Systems Design

Experiment # 11

Implementation of Mealy State Machine

1. Objectives:

- State Machine as a Controller
- Design of Mealy State Machine
 - Overlapping and Non-Overlapping
- Example
- Lab Tasks

2. Theory

A state machine is an abstract way of describing the behavior of a controller. The state machine description reduces the description of any controller to just four things: inputs, outputs, states, and state transition rules. The state machine description contains no details about the implementation of the controller (no flowcharts, programs, schematics, mechanical diagrams, etc.). Instead, a state machine only describes the different states that the controller can be in, the outputs in each state, and what input conditions will cause transitions between states. A state machine description can be used to describe and analyze existing systems and to aid in the design of new systems. The simplified description represented by a state machine allows us to verify proper operation and document the logical operation of a controller before proceeding with detailed design (e.g. writing a program or designing hardware).

3. Design of State Machine

There are four steps to designing a state machine:

- define the outputs – these are the output signals used by the controller
- define the inputs – the input signals provided to the controller
- define all of the possible states that the controller will need to be in and what values the outputs will take for each of these states
- define the exact input conditions that will cause the state machine to switch from one state to another

3.1 Inputs and Outputs

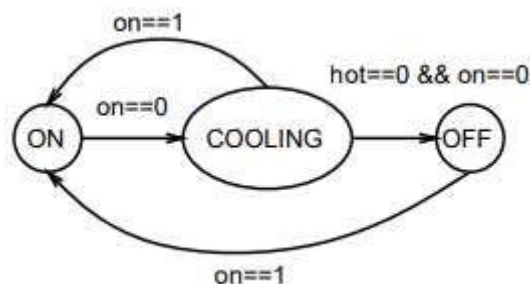
A controller's inputs are typically signals from sensors such as switches, thermometers, pressure gauges, and so on. The outputs are signals that drive actuators such as motors, valves, displays, etc. We will restrict ourselves to inputs and outputs that are binary values (represented as 0/1, false/true, or low/high). To simplify the state machine description, each input and output is given an abbreviation, usually mnemonic. For example an input that indicates that a tank is full might be given the label full, an output that turns a drill motor on might be given the label drill.

3.2 States

A state machines (also called a Finite State Machine or FSM) is a controller that can be in a finite number of different conditions. Each of these conditions is called a state. For example, a simple controller for a motor may have two states: motor turned on and motor turned off. A more complicated controller may have more states. For example, a controller for an overhead projector cooling fan motor (which must continue to run until it the bulb has cooled down, even if the projector is turned off) may have three states: motor running, motor cooling down, and motor off. The number of states required to describe the state machine and the definitions of each of these states will be determined by the application. Each state is given a mnemonic symbol (ON, COOLING, OFF etc).

3.3 State Transition Diagram

The rules describing the transitions between states are commonly given as a diagram called a state transition diagram (or simply "state diagram"). In the state diagram each state is represented by a circle. Each circle is labeled with its state name. Lines are drawn to show the possible transitions between the states and each such line is labelled with a logical expression describing the input conditions required for that transition to happen. If none of the transition conditions is met, the controller remains in its current state. For example, the 3-state motor controller described above might have a switch input (on) and an input from a temperature sensor (hot). The state transition diagram might look as follows:



There are many different conventions for drawing state diagrams. For example, it's common to also show the values of the outputs on the state diagram.

Overlap: The final bits of one sequence can be the start of another sequence.

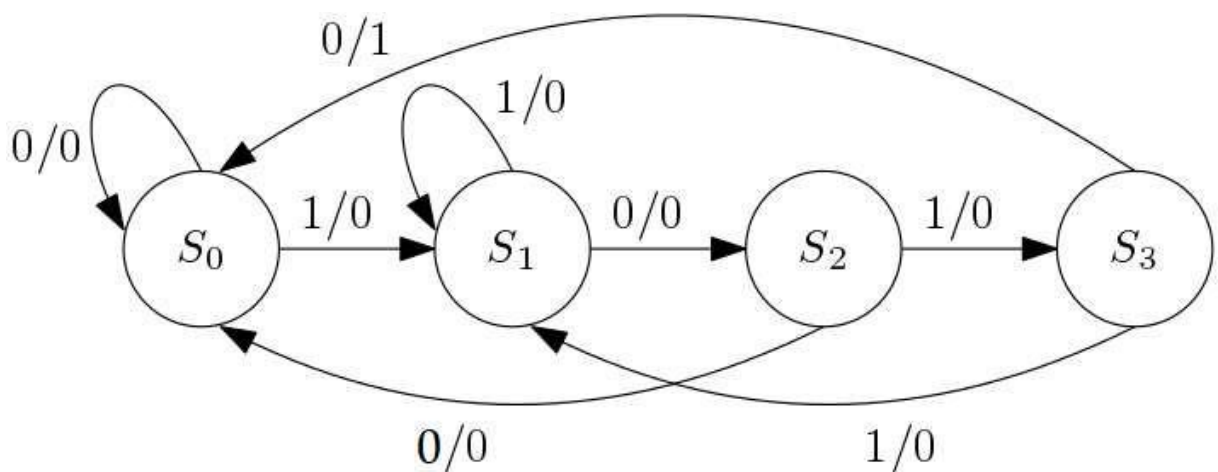
Non-Overlap: The detector resets itself to the start state when the sequence has been detected.

In an overlapping sequence detector, the last bit of one sequence becomes the first bit of the next sequence. However, in a non-overlapping sequence detector, the last bit of one sequence does not become the first bit of the next sequence.

4. Example

- a. You are required to design a controller for a system that actuates on the input provided by a 1-bit sensor. The system actuates (gives O/P as 1) when it detects two consecutive ones followed by a zero which are then followed by two consecutive ones.

1011



Verilog Code: -

```

module mealy(din,clk,reset,y);
  input din;
  input clk;
  input reset;
  output reg y;

  reg [1:0] cst, nst;

  parameter S0 = 2'b00,
            S1 = 2'b01,
            S2 = 2'b10,
            S3 = 2'b11;
  always@(posedge clk)
  begin
    if (reset)
      cst <= S0;
    else
      cst <= nst;
    end
  
```

```
always @(cst or din)
begin
case (cst)
S0: if (din == 1'b1)
begin
nst = S1;
y=1'b0;
end
else
begin
nst = cst;
y=1'b0;
end
S1: if (din == 1'b0)
begin
nst = S2;
y=1'b0;
end
else
begin
y=1'b0;
nst = cst;
end
S2: if (din == 1'b1)
begin
nst = S3;
y=1'b0;
end
else
begin
nst = S0;
y=1'b0;
end
S3: if (din == 1'b0)
begin
nst = S0;
y=1'b1;
end
else
begin
nst = S1;
y=1'b0;
end
default: nst = S0;
endcase
end
endmodule
```

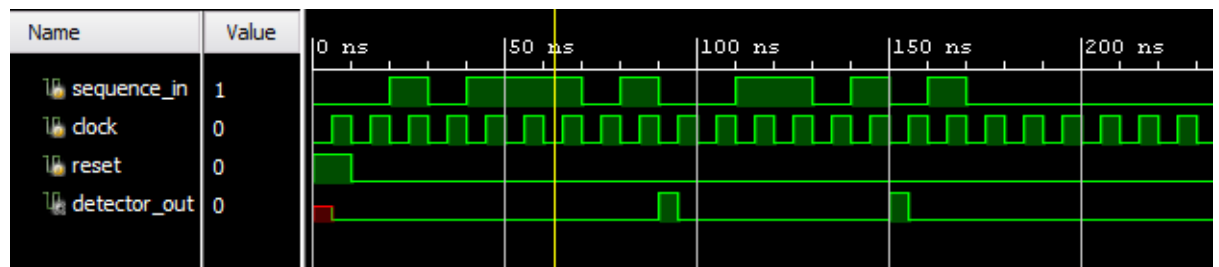
Test Bench: -

```
module TestBench();
reg sequence_in;
reg clock;
```

```
reg reset;

// Outputs
wire detector_out;

// Instantiate the Sequence Detector using Moore FSM
mealy uut (
    .din(sequence_in),
    .clk(clock),
    .reset(reset),
    .y(detector_out)
);
initial begin
    clock = 0;
    forever #5 clock = ~clock;
end
initial begin
    // Initialize Inputs
    sequence_in = 0;
    reset = 1;
    // Wait 100 ns for global reset to finish
    #10;
    reset = 0;
    #10;
    sequence_in = 1;
    #10;
    sequence_in = 0;
    #10;
    sequence_in = 1;
    #30;
    sequence_in = 0;
    #10;
    sequence_in = 1;
    #10;
    sequence_in = 0;
    #10;
    sequence_in = 0;
    #10;
    sequence_in = 1;
    #10;
    sequence_in = 1;
    #10
    sequence_in = 0;
    #10;
    sequence_in = 1;
    #10;
    sequence_in = 0;
    #10;
    sequence_in = 1;
    #10
    sequence_in = 0;
    end
endmodule
```

Output: -

5. Task

1. Execute the code, given in example for overlapping/non-overlapping case.
2. You are required to write code for a basic sequential state machine using mealy based design, which detects **11011** for the following cases:
 - a. Overlapping
 - b. Non-Overlapping

(Draw complete state diagram and elaborate the table for previous, current and next state respectively)